Creational

Singleton

```
public class Singleton {

    private static Singleton instance;

    private Singleton(){}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                instance = new Singleton();
            }
        }
        return instance;
    }

    public static void main(String[] args) {
        Singleton singleton = Singleton.getInstance();
        System.out.println(singleton);
    }

}
```

Builder

```
public class Vehicle {

    private String vin;
    private String color;

    public String getVin() {
        return vin;
    }

    public String getColor() {
        return color;
    }

    public Vehicle(VehicleBuilder builder) {
        this.vin = builder.vin;
        this.color = builder.color;
    }
```

```
    public static class VehicleBuilder {
        private String vin;
        private String color;

        public VehicleBuilder(String vin) {
            this.vin = vin;
        }

        public VehicleBuilder setColor(String color) {
            this.color = color;
            return this;
        }

        public Vehicle build() {
            return new Vehicle(this);
        }
    }


    public static void main(String[] args) {
        Vehicle vehicle = new Vehicle
                .VehicleBuilder("1GNEK13ZX4R118208")
                .setColor(RED.toString()).build();
        System.out.println(String.format("%s : %s",
                vehicle.getVin(), vehicle.getColor()));
    }

}
```

Factory

```
import lombok.Data;

@Data
public class VehicleContext {
    private String id;
}
```

```
@Data
public class CarContext extends VehicleContext {
}
```

```java
@Data
public class BikeContext extends VehicleContext {
}
```

```java
@Component
public class VehicleBuilder {

    private static final String bikeId = "1GNEK13ZX4R118208";

    public BikeContext buildBikeContext(){
        BikeContext bikeContext = new BikeContext();
        bikeContext.setId(bikeId);
        return bikeContext;
    }

}
```

```java
public enum VehicleType {
    BIKE,
    CAR
}
```

```java
public interface Vehicle<T extends VehicleContext> {
    int getWheel();
    VehicleType getType();
    void process(T context);
}
```

```java
@Service
@RequiredArgsConstructor
public class VehicleService {

    private final VehicleProvider vehicleProvider;
    private final VehicleBuilder vehicleBuilder;

    public void process(){
        Vehicle vehicle = vehicleProvider.getVehicle(VehicleType.BIKE);
        System.out.println("Vehicle's Wheel: " + vehicle.getWheel());
        vehicle.process(vehicleBuilder.buildBikeContext());
    }

}
```

```java
@Service
@Slf4j
@RequiredArgsConstructor
public class Bike implements Vehicle<BikeContext> {

    private static final VehicleType type = VehicleType.BIKE;
    private int wheel;

    public Bike(int wheel) {
        this.wheel = wheel;
    }

    @Override
    public int getWheel() {
        return wheel;
    }

    @Override
    public VehicleType getType() {
        return type;
    }

    @Override
    public void process(BikeContext bikeContext) {
       log.info(type + " : process");
    }

}

@Service
@Slf4j
@RequiredArgsConstructor
public class Car implements Vehicle<CarContext> {

    private static final VehicleType type = VehicleType.CAR;
    private int wheel;

    public Car(int wheel) {
        this.wheel = wheel;
    }

    @Override
    public int getWheel() {
        return wheel;
    }

    @Override
    public VehicleType getType() {
        return type;
    }

    @Override
    public void process(CarContext context) {
       log.info(type + " :process");
    }
}
```

```java
@Service
@Slf4j
public class VehicleProvider {

   private Map<VehicleType, Vehicle> vehicleMap;

   @Autowired
   public VehicleProvider(List<Vehicle> vehicleList){
      vehicleMap = new HashMap<>();
      for(Vehicle vehicle : vehicleList){
         vehicleMap.put(vehicle.getType(), vehicle);
      }
   }

   public Vehicle getVehicle(VehicleType type){
      return vehicleMap.get(type);
   }

}
```

Abstract Factory

```java
public abstract class AbstractFactory implements Shape {

    @Override
    public void process(){
       preProcess();
       handleProcess();
       postProcess();
    }

    abstract protected void preProcess();
    abstract protected void handleProcess();
    abstract protected void postProcess();

}
```

```java
public interface Shape {
   void process();
   ShapeType getType();
}
```

```java
@Service
public class Rectangle extends AbstractFactory {

    @Override
    protected void preProcess() {
        System.out.println(getType() + " :preProcess");
    }

    @Override
    protected void handleProcess() {
        System.out.println(getType() + " :handleProcess");
    }

    @Override
    protected void postProcess() {
        System.out.println(getType() + " :postProcess");
    }

    @Override
    public ShapeType getType() {
        return ShapeType.RECTANGLE;
    }
}
```

```java
@Service
public class RoundedRectangle extends AbstractFactory {

    @Override
    public ShapeType getType() {
        return ShapeType.ROUNDED_RECTANGLE;
    }

    @Override
    protected void preProcess() {

    }

    @Override
    protected void handleProcess() {

    }

    @Override
    protected void postProcess() {

    }
}
```

```java
@Service
public class Square extends AbstractFactory {

    @Override
    public void process() {
        System.out.println("Square");
    }

    @Override
    protected void preProcess() {

    }

    @Override
    protected void handleProcess() {

    }

    @Override
    protected void postProcess() {

    }

    @Override
    public ShapeType getType() {
        return ShapeType.SQUARE;
    }

}
```

```java
@Service
public class RoundedSquare extends AbstractFactory {

    @Override
    public ShapeType getType() {
        return ShapeType.ROUNDED_SQUARE;
    }

    @Override
    protected void preProcess() {

    }

    @Override
    protected void handleProcess() {

    }

    @Override
    protected void postProcess() {

    }
}
```

```java
public enum ShapeType {
    ROUNDED_RECTANGLE,
    RECTANGLE,
    ROUNDED_SQUARE,
    SQUARE
}
```

```java
@Service
public class FactoryProvider {

    private Map<ShapeType, Shape> shapeProviders;

    @Autowired
    public FactoryProvider(List<Shape> shapeList) {
        shapeProviders = new HashMap<>();
        for (Shape shape : shapeList) {
            shapeProviders.put(shape.getType(), shape);
        }
    }

    public Shape getShape(ShapeType shapeType){
        return shapeProviders.get(shapeType);
    }

}
```

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    private final FactoryProvider factoryProvider;

    @PostConstruct
    public void process() {
        factoryProvider.getShape(ShapeType.RECTANGLE).process();
    }

}
```

Prototype

```
@Data
public abstract class AbstractVehicle {
   private List<String> vehicleList;
   protected abstract AbstractVehicle clone();
}
```

```
@Data
@AllArgsConstructor
public class Car extends AbstractVehicle {

    private List<String> vehicleList;

    @Override
    public AbstractVehicle clone() {
        return new Car(vehicleList);
    }
}
```

```
@Component
@RequiredArgsConstructor
public class Monitor {


    @PostConstruct
    public void process() {
        System.out.println(new Car(new ArrayList<>()).clone());
    }

}
```

Object Pool (Apache Commons Pool Can Be Used Here)

```
@Component
@RequiredArgsConstructor
public class Monitor {

    private final DatabaseService databaseService;

    @PostConstruct
    public void process() {
        databaseService.executeQuery("SELECT * FROM users");
    }

}
```

```java
@Service
@RequiredArgsConstructor
public class DatabaseService {

    private final ConnectionPoolManager connectionPoolManager;

    public void executeQuery(String query) {
        Connection connection = null;
        try {
            connection = connectionPoolManager.borrowConnection();
            connection.executeQuery(query);
        } finally {
            if (connection != null) {
                connectionPoolManager.returnConnection(connection);
            }
        }
    }
}
```

```java
@Component
public class ConnectionPoolManager {
    private static final int MAX_POOL_SIZE = 5;

    private Queue<Connection> pool;

    public ConnectionPoolManager() {
        pool = new ConcurrentLinkedQueue<>();
        for (int i = 0; i < MAX_POOL_SIZE; i++) {
            pool.add(createConnection("Connection-" + (i + 1)));
        }
    }

    public Connection borrowConnection() {
        if (pool.isEmpty()) {
            throw new RuntimeException("No available connections in the pool");
        }
        return pool.poll();
    }

    public void returnConnection(Connection connection) {
        if (pool.size() >= MAX_POOL_SIZE) {
            throw new RuntimeException("Pool is full, cannot return connection");
        }
        pool.add(connection);
    }

    private Connection createConnection(String id) {
        return new Connection(id);
    }
}
```

```java
public class Connection {
    private String id;

    public Connection(String id) {
        this.id = id;
    }

    public void executeQuery(String query) {
        System.out.println("Executing query '" + query + "' on connection " + id);
    }
}
```

Adapter

```
public interface WebDriver {
    void getElement();
    void selectElement();
}
```

```
public class ChromeDriver implements WebDriver {
    @Override
    public void getElement() {
        System.out.println("ChromeDriver getElement");
    }

    @Override
    public void selectElement() {
        System.out.println("ChromeDriver selectElement");
    }
}
```

```
public class IEDriver {

    public void findElement() {
        System.out.println("IEDriver findElement");
    }

    public void clickElement() {
        System.out.println("IEDriver clickElement");
    }
}
```

```java
public class WebDriverAgent implements WebDriver {

    private IEDriver ieDriver;

    public WebDriverAgent(IEDriver ieDriver){
        this.ieDriver = ieDriver;
    }

    @Override
    public void getElement() {
        ieDriver.findElement();
    }

    @Override
    public void selectElement() {
        ieDriver.clickElement();
    }
}
```

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() {
        WebDriver w = new ChromeDriver();
        w.getElement();
        w.selectElement();

        IEDriver e = new IEDriver();
        e.findElement();
        e.clickElement();

        WebDriver w1 = new WebDriverAgent(e);
        w1.getElement();
        w1.selectElement();
    }

}
```

Bridge (Adapter Of Adapter)

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() {

        TV sonyNewRemote = new Sony(new NewRemote());
        sonyNewRemote.on();
        sonyNewRemote.off();
        System.out.println();

        TV philipsOldRemote = new Philips(new OldRemote());
        philipsOldRemote.on();
        philipsOldRemote.off();
    }

}
```

```
abstract public class TV {
    Remote remote;

    TV(Remote r){
        this.remote = r;
    }

    abstract public void on();
    abstract public void off();
}
```

```
public interface Remote {
     void on();
     void off();
}
```

```java
public class Sony extends TV {

    Remote remote;

    public Sony(Remote r){
        super(r);
        this.remote = r;
    }

    @Override
    public void on() {
        System.out.println("Sony on");
        remote.on();
    }

    @Override
    public void off() {
        System.out.println("Sony off");
        remote.off();
    }
}
```

```java
public class Philips extends TV {

    Remote remote;

    public Philips(Remote r){
        super(r);
        this.remote = r;
    }

    @Override
    public void on() {
        System.out.println("Sony on");
        remote.on();
    }

    @Override
    public void off() {
        System.out.println("Sony off");
        remote.off();
    }
}
```

```java
public class NewRemote implements Remote {
    @Override
    public void on() {
        System.out.println("NewRemote on");
    }

    @Override
    public void off() {
        System.out.println("NewRemote off");
    }
}
```

```java
public class OldRemote implements Remote {
    @Override
    public void on() {
        System.out.println("OldRemote on");
    }

    @Override
    public void off() {
        System.out.println("OldRemote off");
    }
}
```

Filter or Criteria

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() {
        List<Person> persons = new ArrayList<>();

        persons.add(new Person("Robert", "Male", "Single"));
        persons.add(new Person("John", "Male", "Married"));
        persons.add(new Person("Diana", "Female", "Single"));
        persons.add(new Person("Laura", "Female", "Married"));

        Criteria male = new CriteriaMale();
        Criteria female = new CriteriaFemale();
        Criteria single = new CriteriaSingle();
        Criteria singleMale = new AndCriteria(single, male);
        Criteria singleOrFemale = new OrCriteria(single, female);

        System.out.println("Males: " + male.meetCriteria(persons).toString());
        System.out.println("Females: " + female.meetCriteria(persons));
        System.out.println("Single Males: " + singleMale.meetCriteria(persons));
        System.out.println("Single Or Females: " + singleOrFemale.meetCriteria(persons));
    }

}
```

```java
@Data
@AllArgsConstructor
public class Person {
    private String name;
    private String gender;
    private String maritalStatus;
}
```

```java
public interface Criteria {
    List<Person> meetCriteria(List<Person> persons);
}
```

```java
public class CriteriaMale implements Criteria {

    @Override
    public List<Person> meetCriteria(List<Person> persons) {
        List<Person> malePersons = new ArrayList<Person>();

        for (Person person : persons) {
            if(person.getGender().equalsIgnoreCase("MALE")){
                malePersons.add(person);
            }
        }
        return malePersons;
    }
}
```

```java
public class CriteriaFemale implements Criteria {

    @Override
    public List<Person> meetCriteria(List<Person> persons) {
        List<Person> femalePersons = new ArrayList<Person>();

        for (Person person : persons) {
            if(person.getGender().equalsIgnoreCase("FEMALE")){
                femalePersons.add(person);
            }
        }
        return femalePersons;
    }
}
```

```java
public class CriteriaSingle implements Criteria {

    @Override
    public List<Person> meetCriteria(List<Person> persons) {
        List<Person> singlePersons = new ArrayList<Person>();

        for (Person person : persons) {
            if(person.getMaritalStatus().equalsIgnoreCase("SINGLE")){
                singlePersons.add(person);
            }
        }
        return singlePersons;
    }

}
```

```java
public class AndCriteria implements Criteria {

    private Criteria criteria;
    private Criteria otherCriteria;

    public AndCriteria(Criteria criteria, Criteria otherCriteria) {
        this.criteria = criteria;
        this.otherCriteria = otherCriteria;
    }

    @Override
    public List<Person> meetCriteria(List<Person> persons) {

        List<Person> firstCriteriaPersons = criteria.meetCriteria(persons);
        return otherCriteria.meetCriteria(firstCriteriaPersons);
    }
}
```

```java
public class OrCriteria implements Criteria {

    private Criteria criteria;
    private Criteria otherCriteria;

    public OrCriteria(Criteria criteria, Criteria otherCriteria) {
        this.criteria = criteria;
        this.otherCriteria = otherCriteria;
    }

    @Override
    public List<Person> meetCriteria(List<Person> persons) {
        List<Person> firstCriteriaItems = criteria.meetCriteria(persons);
        List<Person> otherCriteriaItems = otherCriteria.meetCriteria(persons);

        for (Person person : otherCriteriaItems) {
            if(!firstCriteriaItems.contains(person)){
                firstCriteriaItems.add(person);
            }
        }
        return firstCriteriaItems;
    }
}
```

Proxy

```
class DatabaseExecutorProxy implements DatabaseExecutor {
    boolean ifAdmin;
    DatabaseExecutorImpl dbExecuter;

    public DatabaseExecutorProxy(String name, String passwd) {
        if(name == "Admin" && passwd == "Admin@123") {
            ifAdmin = true;
        }
        dbExecuter = new DatabaseExecutorImpl();
    }

    @Override
    public void executeDatabase(String query) throws Exception {
        if(ifAdmin) {
            dbExecuter.executeDatabase(query);
        } else {
            if(query.equals("DELETE")) {
                throw new Exception("DELETE not allowed for non-admin user");
            } else {
                dbExecuter.executeDatabase(query);
            }
        }
    }
}
```

```
public interface DatabaseExecutor {
    public void executeDatabase(String query) throws Exception;
}
```

```
public class DatabaseExecutorProxy implements DatabaseExecutor {
    boolean ifAdmin;
    DatabaseExecutorImpl dbExecuter;

    public DatabaseExecutorProxy(String name, String passwd) {
        if(name == "Admin" && passwd == "Admin@123") {
            ifAdmin = true;
        }
        dbExecuter = new DatabaseExecutorImpl();
    }

    @Override
    public void executeDatabase(String query) throws Exception {
        if(ifAdmin) {
            dbExecuter.executeDatabase(query);
        } else {
            if(query.equals("DELETE")) {
                throw new Exception("DELETE not allowed for non-admin user");
            } else {
                dbExecuter.executeDatabase(query);
            }
        }
    }
}
```

```
class DatabaseExecutorImpl implements DatabaseExecutor {

    @Override
    public void executeDatabase(String query) throws Exception {
        System.out.println("Going to execute Query: " + query);
    }

}
```

Flyweight

```
public interface Employee {
    void assignSkill(String skill);
    void task();
}
```

```
@Component
@RequiredArgsConstructor
public class Monitor {

    private static String employeeType[] = {"Developer", "Tester"};
    private static String skills[] = {"Java", "C++", ".Net", "Python"};

    @PostConstruct
    public void process() throws Exception {
        for(int i = 0; i < 10; i++) {
            Employee e = EmployeeFactory.getEmployee(getRandEmployee());
            e.assignSkill(getRandSkill());
            e.task();
        }
    }

    private static String getRandEmployee() {
        Random r = new Random();
        int randInt = r.nextInt(employeeType.length);

        return employeeType[randInt];
    }

    private static String getRandSkill() {
        Random r = new Random();
        int randInt = r.nextInt(skills.length);

        return skills[randInt];
    }

}
```

```java
class Developer implements Employee {

    private final String JOB;
    private String skill;

    public Developer() {
        JOB = "Fix the issue";
    }

    @Override
    public void assignSkill(String skill) {
        this.skill = skill;
    }

    @Override
    public void task() {
        System.out.println("Developer with skill: " + this.skill + " with Job: " + JOB);
    }

}
```

```java
class Tester implements Employee {

    private final String JOB;

    private String skill;

    public Tester() {
        JOB = "Test the issue";
    }

    @Override
    public void assignSkill(String skill) {
        this.skill = skill;
    }

    @Override
    public void task() {
        System.out.println("Tester with Skill: " + this.skill + " with Job: " + JOB);
    }

}
```

```java
public class EmployeeFactory {
    private static HashMap<String, Employee> m = new HashMap<String, Employee>();

    public static Employee getEmployee(String type) {
        Employee employee = null;
        if(m.get(type) != null) {
            employee = m.get(type);
        } else {
            switch(type) {
                case "Developer":
                    System.out.println("Developer Created");
                    employee = new Developer();
                    break;
                case "Tester":
                    System.out.println("Tester Created");
                    employee = new Tester();
                    break;
                default:
                    System.out.println("No Such Employee");
            }

            m.put(type, employee);
        }
        return employee;
    }
}
```

Facade

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        String test = "CheckElementPresent";
        WebExplorerHelperFacade.generateReports("firefox", "html", test);
        WebExplorerHelperFacade.generateReports("firefox", "junit", test);
        WebExplorerHelperFacade.generateReports("chrome", "html", test);
        WebExplorerHelperFacade.generateReports("chrome", "junit", test);
    }

}
```

```
class Firefox {
    public static Driver getFirefoxDriver() {
        return null;
    }

    public static void generateHTMLReport(String test, Driver driver) {
        System.out.println("Generating HTML Report for Firefox Driver");
    }

    public static void generateJUnitReport(String test, Driver driver) {
        System.out.println("Generating JUNIT Report for Firefox Driver");
    }
}
```

```
class Chrome {
    public static Driver getChromeDriver() {
        return null;
    }

    public static void generateHTMLReport(String test, Driver driver) {
        System.out.println("Generating HTML Report for Chrome Driver");
    }

    public static void generateJUnitReport(String test, Driver driver) {
        System.out.println("Generating JUNIT Report for Chrome Driver");
    }
}
```

```
public class WebExplorerHelperFacade {
    public static void generateReports(String explorer, String report, String test) {
        Driver driver = null;
        switch(explorer) {
            case "firefox":
                driver = Firefox.getFirefoxDriver();
                switch(report) {
                    case "html":
                        Firefox.generateHTMLReport(test, driver);
                        break;
                    case "junit":
                        Firefox.generateJUnitReport(test, driver);
                        break;
                }
                break;
            case "chrome":
                driver = Chrome.getChromeDriver();
                switch(report) {
                    case "html":
                        Chrome.generateHTMLReport(test, driver);
                        break;
                    case "junit":
                        Chrome.generateJUnitReport(test, driver);
                        break;
                }
        }
    }
}
```

Decorator or Wrapper

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        Dress sportyDress = new SportyDress(new BasicDress());
        sportyDress.assemble();
        System.out.println();

        Dress fancyDress = new SportyDress(new BasicDress());
        fancyDress.assemble();
        System.out.println();

        Dress casualDress = new CasualDress(new BasicDress());
        casualDress.assemble();
        System.out.println();


        Dress sportyFancyDress = new SportyDress(new FancyDress(new CasualDress(new BasicDress())));
        sportyFancyDress.assemble();
        System.out.println();

        Dress casualFancyDress = new CasualDress(new FancyDress(new BasicDress()));
        casualFancyDress.assemble();
    }

}
```

```java
public interface Dress {
    void assemble();
}
```

```java
class DressDecorator implements Dress {
    protected Dress dress;

    public DressDecorator(Dress c) {
        this.dress = c;
    }

    @Override
    public void assemble() {
        this.dress.assemble();
    }
}
```

```java
public class BasicDress implements Dress {
    @Override
    public void assemble() {
        System.out.println("Basic Dress Features");
    }
}
```

```java
public class CasualDress extends DressDecorator {
    public CasualDress(Dress c) {
        super(c);
    }

    @Override
    public void assemble() {
        super.assemble();
        System.out.println("Adding Casual Dress Features");
    }
}
```

```
public class SportyDress extends DressDecorator {
    public SportyDress(Dress c) {
        super(c);
    }

    @Override
    public void assemble() {
        super.assemble();
        System.out.println("Adding Sporty Dress Features");
    }
}
```

```
public class FancyDress extends DressDecorator {
    public FancyDress(Dress c) {
        super(c);
    }

    @Override
    public void assemble() {
        super.assemble();
        System.out.println("Adding Fancy Dress Features");
    }
}
```

Composite

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        CompositeAccount component = new CompositeAccount();

        component.addAccount(new DepositeAccount("DA001", 100));
        component.addAccount(new DepositeAccount("DA002", 150));
        component.addAccount(new SavingAccount("SA001", 200));

        float totalBalance = component.getBalance();
        System.out.println("Total Balance : " + totalBalance);
    }

}
```

```java
public class CompositeAccount extends Account {
    private float totalBalance;
    private List<Account> accountList = new ArrayList<Account>();

    public float getBalance() {
        totalBalance = 0;
        for (Account f : accountList) {
            totalBalance = totalBalance + f.getBalance();
        }
        return totalBalance;
    }

    public void addAccount(Account acc) {
        accountList.add(acc);
    }

    public void removeAccount(Account acc) {
        accountList.add(acc);
    }
}
```

```java
abstract class Account {
    public abstract float getBalance();
}
```

```java
public class DepositeAccount extends Account {
    private String accountNo;
    private float accountBalance;

    public DepositeAccount(String accountNo, float accountBalance) {
        super();
        this.accountNo = accountNo;
        this.accountBalance = accountBalance;
    }

    public float getBalance() {
        return accountBalance;
    }

}
```

```
public class SavingAccount extends Account {
    private String accountNo;
    private float accountBalance;

    public SavingAccount(String accountNo, float accountBalance) {
        super();
        this.accountNo = accountNo;
        this.accountBalance = accountBalance;
    }

    public float getBalance() {
        return accountBalance;
    }
}
```

Structural

Chain Of Responsibility

```
@Component
@RequiredArgsConstructor
public class Monitor {

    private AbstractLogger getChainOfLoggers(){

        AbstractLogger errorLogger = new ErrorLogger(AbstractLogger.ERROR);
        AbstractLogger fileLogger = new FileLogger(AbstractLogger.DEBUG);
        AbstractLogger consoleLogger = new ConsoleLogger(AbstractLogger.INFO);

        errorLogger.setNextLogger(fileLogger);
        fileLogger.setNextLogger(consoleLogger);

        return errorLogger;
    }

    @PostConstruct
    public void process() throws Exception {
        AbstractLogger loggerChain = getChainOfLoggers();

        loggerChain.logMessage(AbstractLogger.INFO,
                "This is an information.");

        loggerChain.logMessage(AbstractLogger.DEBUG,
                "This is an debug level information.");

        loggerChain.logMessage(AbstractLogger.ERROR,
                "This is an error information.");
    }

}
```

```
public abstract class AbstractLogger {
    public static int INFO = 1;
    public static int DEBUG = 2;
    public static int ERROR = 3;

    protected int level;
    protected AbstractLogger nextLogger;

    public void setNextLogger(AbstractLogger nextLogger){
        this.nextLogger = nextLogger;
    }

    public void logMessage(int level, String message){
        if(this.level <= level){
            write(message);
        }
        if(nextLogger !=null){
            nextLogger.logMessage(level, message);
        }
    }

    abstract protected void write(String message);

}
```

```
public class ConsoleLogger extends AbstractLogger {

    public ConsoleLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("Standard Console::Logger: " + message);
    }
}
```

```
public class ErrorLogger extends AbstractLogger {

    public ErrorLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("Error Console::Logger: " + message);
    }
}
```

```java
public class FileLogger extends AbstractLogger {

    public FileLogger(int level){
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("File::Logger: " + message);
    }
}
```

Command (Action Or Transaction)

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        Stock abcStock = new Stock();
        BuyStock buyStockOrder = new BuyStock(abcStock);
        SellStock sellStockOrder = new SellStock(abcStock);
        Broker broker = new Broker();
        broker.takeOrder(buyStockOrder);
        broker.takeOrder(sellStockOrder);
        broker.placeOrders();
    }

}
```

```java
public interface Order {
    void execute();
}
```

```java
public class Stock {

    private String name = "ABC";
    private int quantity = 10;

    public void buy() {
        System.out.println("Stock [ Name: " + name + ", Quantity: " + quantity + " ] bought");
    }

    public void sell() {
        System.out.println("Stock [ Name: " + name + ", Quantity: " + quantity + " ] sold");
    }
}
```

```java
public class Broker {
    private List<Order> orderList = new ArrayList<Order>();

    public void takeOrder(Order order){
        orderList.add(order);
    }

    public void placeOrders(){

        for (Order order : orderList) {
            order.execute();
        }
        orderList.clear();
    }
}
```

```java
public class BuyStock implements Order {
    private Stock abcStock;

    public BuyStock(Stock abcStock){
        this.abcStock = abcStock;
    }

    public void execute() {
        abcStock.buy();
    }
}
```

```
public class SellStock implements Order {
    private Stock abcStock;

    public SellStock(Stock abcStock){
        this.abcStock = abcStock;
    }

    public void execute() {
        abcStock.sell();
    }
}
```

Iterator

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        NameRepository namesRepository = new NameRepository();

        for(Iterator iter = namesRepository.getIterator(); iter.hasNext();){
            String name = (String)iter.next();
            System.out.println("Name : " + name);
        }
    }

}
```

```
public interface Iterator {
    boolean hasNext();
    Object next();
}
```

```java
public interface Container {
    Iterator getIterator();
}
```

```java
public class NameRepository implements Container {
    public String names[] = {"Robert" , "John" ,"Julie" , "Lora"};

    @Override
    public Iterator getIterator() {
        return new NameIterator();
    }

    private class NameIterator implements Iterator {
        int index;

        @Override
        public boolean hasNext() {
            if(index < names.length){
                return true;
            }
            return false;
        }

        @Override
        public Object next() {
            if(this.hasNext()){
                return names[index++];
            }
            return null;
        }
    }
}
```

Intepreter

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    public static Expression getMaleExpression() {
        Expression robert = new TerminalExpression("Robert");
        Expression john = new TerminalExpression("John");
        return new OrExpression(robert, john);
    }

    public static Expression getMarriedWomanExpression() {
        Expression julie = new TerminalExpression("Julie");
        Expression married = new TerminalExpression("Married");
        return new AndExpression(julie, married);
    }

    @PostConstruct
    public void process() throws Exception {
        Expression isMale = getMaleExpression();
        Expression isMarriedWoman = getMarriedWomanExpression();

        System.out.println("John is male? " + isMale.interpret("John"));
        System.out.println("Julie is a married women? " +
                isMarriedWoman.interpret("Married Julie"));
    }

}
```

```java
public interface Expression {
    boolean interpret(String context);
}
```

```java
public class TerminalExpression implements Expression {

    private String data;

    public TerminalExpression(String data){
        this.data = data;
    }

    @Override
    public boolean interpret(String context) {

        if(context.contains(data)){
            return true;
        }
        return false;
    }
}
```

```java
public class OrExpression implements Expression {

    private Expression expr1 = null;
    private Expression expr2 = null;

    public OrExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    @Override
    public boolean interpret(String context) {
        return expr1.interpret(context) || expr2.interpret(context);
    }
}
```

```
public class AndExpression implements Expression {

    private Expression expr1 = null;
    private Expression expr2 = null;

    public AndExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    @Override
    public boolean interpret(String context) {
        return expr1.interpret(context) && expr2.interpret(context);
    }
}
```

Null Object

```
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        AbstractCustomer customer1 = CustomerFactory.getCustomer("Rob");
        AbstractCustomer customer2 = CustomerFactory.getCustomer("Bob");
        AbstractCustomer customer3 = CustomerFactory.getCustomer("Julie");
        AbstractCustomer customer4 = CustomerFactory.getCustomer("Laura");

        System.out.println("Customers");
        System.out.println(customer1.getName());
        System.out.println(customer2.getName());
        System.out.println(customer3.getName());
        System.out.println(customer4.getName());
    }

}
```

```
public abstract class AbstractCustomer {
    protected String name;
    public abstract boolean isNil();
    public abstract String getName();
}
```

```java
public class RealCustomer extends AbstractCustomer {

    public RealCustomer(String name) {
        this.name = name;
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public boolean isNil() {
        return false;
    }
}
```

```java
public class NullCustomer extends AbstractCustomer {

    @Override
    public String getName() {
        return "Not Available in Customer Database";
    }

    @Override
    public boolean isNil() {
        return true;
    }
}
```

```
public class CustomerFactory {

    public static final String[] names = {"Rob", "Joe", "Julie"};

    public static AbstractCustomer getCustomer(String name){

        for (int i = 0; i < names.length; i++) {
            if (names[i].equalsIgnoreCase(name)){
                return new RealCustomer(name);
            }
        }
        return new NullCustomer();
    }
}
```

Observer

```
 @PostConstruct
 public void process() throws Exception {
     DeliveryData topic = new DeliveryData();

     Observer obj1 = new Seller();
     Observer obj2 = new User();
     Observer obj3 = new DeliveryWarehouseCenter();

     topic.register(obj1);
     topic.register(obj2);
     topic.register(obj3);

     topic.locationChanged();

     topic.unregister(obj3);

     System.out.println();
     topic.locationChanged();
 }
```

```java
interface Subject {
    void register(Observer obj);
    void unregister(Observer obj);
    void notifyObservers();
}
```

```java
public interface Observer {
    void update(String location);
}
```

```java
public class Seller implements Observer {
    private String location;

    @Override
    public void update(String location) {
        this.location = location;
        showLocation();
    }

    public void showLocation() {
        System.out.println("Notification at Seller - Current Location: " + location);
    }
}
```

```java
public class User implements Observer {
    private String location;

    @Override
    public void update(String location) {
        this.location = location;
        showLocation();
    }

    public void showLocation() {
        System.out.println("Notification at User - Current Location: " + location);
    }
}
```

```java
public class DeliveryWarehouseCenter implements Observer {
    private String location;

    @Override
    public void update(String location) {
        this.location = location;
        showLocation();
    }

    public void showLocation() {
        System.out.println("Notification at Warehouse - Current Location: " + location);
    }
}
```

Mediator

```java
@Component
@RequiredArgsConstructor
public class Monitor {

    @PostConstruct
    public void process() throws Exception {
        User robert = new User("Robert");
        User john = new User("John");
        robert.sendMessage("Hi! John!");
        john.sendMessage("Hello! Robert!");
    }

}
```

```java
public class ChatRoom {
    public static void showMessage(User user, String message){
        System.out.println(new Date().toString() + " [" + user.getName() + "] : " + message);
    }
}
```

```
public class User {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public User(String name){
        this.name  = name;
    }

    public void sendMessage(String message){
        ChatRoom.showMessage(this,message);
    }
}
```

Memento

```
    @PostConstruct
    public void process() throws Exception {
        Originator originator = new Originator();
        CareTaker careTaker = new CareTaker();
        originator.setState("State #1");
        originator.setState("State #2");
        careTaker.add(originator.saveStateToMemento());
        originator.setState("State #3");
        careTaker.add(originator.saveStateToMemento());
        originator.setState("State #4");
        System.out.println("Current State: " + originator.getState());
        originator.getStateFromMemento(careTaker.get(0));
        System.out.println("First saved State: " + originator.getState());
        originator.getStateFromMemento(careTaker.get(1));
        System.out.println("Second saved State: " + originator.getState());
    }
```

```java
public class Originator {
    private String state;

    public void setState(String state){
        this.state = state;
    }

    public String getState(){
        return state;
    }

    public Memento saveStateToMemento(){
        return new Memento(state);
    }

    public void getStateFromMemento(Memento memento){
        state = memento.getState();
    }
}
```

```java
public class Memento {
    private String state;

    public Memento(String state){
        this.state = state;
    }

    public String getState(){
        return state;
    }
}
```

```java
public class CareTaker {
    private List<Memento> mementoList = new ArrayList<>();

    public void add(Memento state){
        mementoList.add(state);
    }

    public Memento get(int index){
        return mementoList.get(index);
    }
}
```

Template

```java
public abstract class Game {
    abstract void initialize();
    abstract void startPlay();
    abstract void endPlay();

    public final void play(){
        initialize();
        startPlay();
        endPlay();
    }
}
```

```java
    @PostConstruct
    public void process() throws Exception {
        Game game = new Cricket();
        game.play();
        System.out.println();
        game = new Football();
        game.play();
    }
```

```java
public class Cricket extends Game {

    @Override
    void endPlay() {
        System.out.println("Cricket Game Finished!");
    }

    @Override
    void initialize() {
        System.out.println("Cricket Game Initialized! Start playing.");
    }

    @Override
    void startPlay() {
        System.out.println("Cricket Game Started. Enjoy the game!");
    }
}
```

```java
public class Football extends Game {

    @Override
    void endPlay() {
        System.out.println("Football Game Finished!");
    }

    @Override
    void initialize() {
        System.out.println("Football Game Initialized! Start playing.");
    }

    @Override
    void startPlay() {
        System.out.println("Football Game Started. Enjoy the game!");
    }
}
```

Visitor

```
    @PostConstruct
    public void process() throws Exception {
        ConcreteVisitableA visitableA = new ConcreteVisitableA();
        ConcreteVisitableB visitableB = new ConcreteVisitableB();

        Visitor visitor = new ConcreteVisitor();
        visitableA.accept(visitor);
        visitableB.accept(visitor);
    }
```

```
interface Visitable {
    void accept(Visitor visitor);
}
```

```
public interface Visitor {
    void visit(ConcreteVisitableA visitable);
    void visit(ConcreteVisitableB visitable);
}
```

```
public class ConcreteVisitor implements Visitor {
    @Override
    public void visit(ConcreteVisitableA visitable) {
        visitable.operationA();
    }

    @Override
    public void visit(ConcreteVisitableB visitable) {
        visitable.operationB();
    }
}
```

```
public class ConcreteVisitableB implements Visitable {
    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }

    public void operationB() {
        System.out.println("Operation B performed by ConcreteVisitableB");
    }
}
```

```
public class ConcreteVisitableA implements Visitable {
    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }

    public void operationA() {
        System.out.println("Operation A performed by ConcreteVisitableA");
    }
}
```

State

```
    @PostConstruct
    public void process() throws Exception {
        Context context = new Context();
        StartState startState = new StartState();
        startState.doAction(context);
        System.out.println(context.getState().toString());
        StopState stopState = new StopState();
        stopState.doAction(context);
        System.out.println(context.getState().toString());
    }
```

```
public interface State {
    void doAction(Context context);
}
```

```java
public class StartState implements State {

    public void doAction(Context context) {
        System.out.println("Player is in start state");
        context.setState(this);
    }

    public String toString(){
        return "Start State";
    }
}
```

```java
public class StopState implements State {

    public void doAction(Context context) {
        System.out.println("Player is in stop state");
        context.setState(this);
    }

    public String toString(){
        return "Stop State";
    }
}
```

```
public class Context {
    private State state;

    public Context(){
        state = null;
    }

    public void setState(State state){
        this.state = state;
    }

    public State getState(){
        return state;
    }
}
```

Strategy

```
    @PostConstruct
    public void process() throws Exception {
        Context context = new Context(new OperationAdd());
        System.out.println("10 + 5 = " + context.executeStrategy(10, 5));
        context = new Context(new OperationSubtract());
        System.out.println("10 - 5 = " + context.executeStrategy(10, 5));
        context = new Context(new OperationMultiply());
        System.out.println("10 * 5 = " + context.executeStrategy(10, 5));
    }
```

```java
public interface Strategy {
    int doOperation(int num1, int num2);
}
```

```java
public class OperationAdd implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 + num2;
    }
}
```

```java
public class OperationSubtract implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 - num2;
    }
}
```

```java
public class OperationMultiply implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 * num2;
    }
}
```

```java
public class Context {
    private Strategy strategy;

    public Context(Strategy strategy){
        this.strategy = strategy;
    }

    public int executeStrategy(int num1, int num2){
        return strategy.doOperation(num1, num2);
    }
}
```